
Système de Recommandation

Dataset ModCloth

Comparaison du Filtrage Collaboratif et du Filtrage Basé sur le
Contenu

Réalisé par :

Khaldoun TAKTAK
Nermine BACHA

Télécom SudParis

Novembre 2025

Table des matières

1	Introduction	2
1.1	Contexte du Projet	2
1.2	Objectifs	2
1.3	Dataset ModCloth	2
2	Problématique	2
3	Méthodologie	3
3.1	Pipeline de Travail	3
3.2	Prétraitement des Données	3
3.2.1	Chargement	3
3.2.2	Nettoyage	3
3.2.3	Filtrage de la Sparsité	3
3.3	Split Train/Test	4
3.4	Filtrage Collaboratif (CF)	4
3.4.1	Approche Choisie	4
3.4.2	Algorithme de Prédiction	4
3.5	Filtrage Basé sur le Contenu	4
3.5.1	Construction des Profils d'Items	4
3.5.2	Vectorisation TF-IDF	4
3.5.3	Profils Utilisateurs	5
3.6	Métriques d'Évaluation	5
3.6.1	Métriques de Prédiction (CF uniquement)	5
3.6.2	Métriques de Recommandation	5
4	Résultats Expérimentaux	6
4.1	Statistiques du Dataset Après Prétraitement	6
4.2	Distribution des Données	6
4.3	Performances du Filtrage Collaboratif	6
4.4	Performances du Filtrage par Contenu	7
4.5	Comparaison des Modèles	7
5	Analyse et Interprétation	7
5.1	Pourquoi le Contenu Gagne	7
5.2	Limites du CF	7
5.3	Performances Absolues	8
6	Problèmes Rencontrés	8
7	Pistes d'Amélioration	8
8	Conclusion	8
8.1	Résultats Principaux	8
8.2	Statistiques Finales du Projet	9
8.3	Limites Identifiées	9

1 Introduction

1.1 Contexte du Projet

Les systèmes de recommandation sont devenus omniprésents dans le commerce électronique moderne. Dans le secteur de la mode en ligne, recommander les bons vêtements aux clients est crucial pour améliorer l'expérience utilisateur et augmenter les ventes. Ce projet s'inscrit dans le cadre d'une étude comparative de deux approches fondamentales de recommandation appliquées au domaine de la mode en ligne.

1.2 Objectifs

L'objectif principal de ce projet est de construire et de comparer deux systèmes de recommandation sur le dataset ModCloth Clothing Fit :

1. **Filtrage Collaboratif (CF)** : basé sur les similarités entre utilisateurs ou items à partir des notes données
2. **Filtrage Basé sur le Contenu** : basé sur les caractéristiques textuelles et catégorielles des items

Ce travail vise à déterminer quelle approche est la plus adaptée pour recommander des vêtements dans un contexte de données réelles avec forte sparsité.

1.3 Dataset ModCloth

Le dataset ModCloth Clothing Fit provient de l'UC San Diego McAuley Lab. Il contient des avis d'utilisateurs sur des vêtements avec :

- Des notes d'évaluation (quality)
- Des informations sur l'ajustement (fit : small, fit, large)
- Des catégories de vêtements
- Des avis textuels (review_text, review_summary)
- Des métadonnées morphologiques (taille, poids, etc.)

2 Problématique

Le défi principal de ce projet est de recommander des vêtements dans un contexte difficile. Le dataset ModCloth présente une sparsité extrême : la plupart des utilisateurs n'ont noté que quelques articles sur des milliers disponibles. C'est comme essayer de deviner les goûts de quelqu'un qui n'a commenté que 5 films sur Netflix.

Cette sparsité pose plusieurs problèmes concrets :

- Difficile de trouver des utilisateurs ayant des goûts similaires (trop peu d'overlaps)
- Impossible de recommander aux nouveaux utilisateurs (pas d'historique)
- Les algorithmes classiques de filtrage collaboratif peinent à trouver des patterns

Notre question est simple : face à ces données réelles et imparfaites, quelle méthode marche le mieux ? Le filtrage collaboratif qui se base sur les similarités entre utilisateurs, ou le filtrage par contenu qui utilise les descriptions des vêtements (catégorie, ajustement, avis textuels) ?

3 Méthodologie

3.1 Pipeline de Travail

1. **Découverte des données** : exploration et compréhension du dataset
2. **Nettoyage** : gestion des valeurs manquantes et sélection des colonnes
3. **Filtrage de la sparsité** : élimination des utilisateurs/items rares
4. **Construction du dataset** : création des indices et split train/test
5. **Modélisation** : implémentation de CF et filtrage par contenu
6. **Évaluation** : calcul de métriques multiples
7. **Comparaison** : analyse des performances relatives

3.2 Prétraitement des Données

3.2.1 Chargement

Le dataset est téléchargé directement depuis l'URL officielle :

```
1 DATA_URL = "https://mcauleylab.ucsd.edu/public_datasets/data/  
  modcloth/modcloth_final_data.json.gz"  
2 df_raw = pd.read_json(DATA_URL, lines=True, compression='gzip')
```

3.2.2 Nettoyage

Trois stratégies de nettoyage ont été appliquées :

- **Colonnes essentielles** (user_id, item_id, quality) : suppression des lignes avec valeurs manquantes
- **Colonnes textuelles** (review_text, review_summary) : remplacement par chaînes vides
- **Colonnes catégorielles** (category, fit) : remplacement par "unknown"

3.2.3 Filtrage de la Sparsité

Pour réduire la sparsité, nous avons appliqué un filtrage itératif conservant uniquement les utilisateurs et items avec au moins 5 interactions :

```
1 MIN_INTERACTIONS = 5  
2 # Filtrage itératif jusqu'à convergence  
3 while True:  
4     user_counts = df['user_id'].value_counts()  
5     item_counts = df['item_id'].value_counts()  
6     valid_users = user_counts[user_counts >= MIN_INTERACTIONS].  
       index  
7     valid_items = item_counts[item_counts >= MIN_INTERACTIONS].  
       index  
8     df = df[df['user_id'].isin(valid_users)]  
9     df = df[df['item_id'].isin(valid_items)]  
10    # Verifier convergence  
11    if len(df) == prev_size: break
```

3.3 Split Train/Test

Un split stratifié par utilisateur (80/20) a été effectué pour simuler un scénario réaliste de prédiction temporelle :

```

1 for user_idx in range(len(unique_users)):
2     user_interactions = ratings_df[ratings_df['user_idx'] ==
        user_idx].index
3     np.random.shuffle(user_interactions)
4     split_point = int(len(user_interactions) * 0.8)
5     train_indices.extend(user_interactions[:split_point])
6     test_indices.extend(user_interactions[split_point:])

```

Données d'entraînement : 12,013 interactions (80%) pour construire les modèles, 4,033 interactions (20%) pour tester. Cette approche stratifiée par utilisateur simule un scénario réaliste de prédiction.

3.4 Filtrage Collaboratif (CF)

3.4.1 Approche Choisie

Nous avons implémenté un **CF item-based avec k-NN** et similarité cosinus :

- Construction d'une matrice sparse utilisateur-item
- Utilisation de k=20 plus proches voisins
- Prédiction par moyenne pondérée des similarités

3.4.2 Algorithme de Prédiction

Pour prédire la note qu'un utilisateur u donnerait à un item i :

$$\hat{r}_{u,i} = \frac{\sum_{j \in N(i)} \text{sim}(i, j) \cdot r_{u,j}}{\sum_{j \in N(i)} \text{sim}(i, j)} \quad (1)$$

où $N(i)$ sont les k items les plus similaires à i que l'utilisateur u a notés, et $\text{sim}(i, j)$ est la similarité cosinus entre items i et j .

Fonctionnement : La prédiction est la moyenne pondérée des notes des k items similaires. Si aucun voisin n'est trouvé, on retourne la note moyenne globale de l'utilisateur.

3.5 Filtrage Basé sur le Contenu

3.5.1 Construction des Profils d'Items

Pour chaque item, nous avons concaténé :

- Les avis textuels (review_text, review_summary)
- La catégorie (répétée 3 fois pour amplifier son importance)
- Le fit (répété 2 fois)

3.5.2 Vectorisation TF-IDF

Transformation des textes en vecteurs TF-IDF :

```

1 tfidf = TfidfVectorizer(
2     max_features=1000,
3     stop_words='english',
4     ngram_range=(1, 2),
5     min_df=2,
6     max_df=0.8
7 )
8 item_content_matrix = tfidf.fit_transform(item_text_list)

```

3.5.3 Profils Utilisateurs

Le profil d'un utilisateur u est calculé comme la moyenne pondérée des vecteurs TF-IDF des items qu'il a bien notés :

$$\vec{p}_u = \frac{\sum_{i \in I_u} r_{u,i} \cdot \vec{v}_i}{\sum_{i \in I_u} r_{u,i}} \quad (2)$$

où I_u sont les items notés ≥ 4 par l'utilisateur u , $r_{u,i}$ est la note, et $\vec{v}_i$ est le vecteur TF-IDF de l'item i .

Construction du profil : Chaque item est représenté par le TF-IDF de ses textes d'avis, avec amplification de la catégorie ($\times 3$) et du fit ($\times 2$) pour renforcer ces attributs structurés. Le profil utilisateur devient la moyenne pondérée des items qu'il a bien notés.

3.6 Métriques d'Évaluation

3.6.1 Métriques de Prédiction (CF uniquement)

— **RMSE** (Root Mean Squared Error) :

$$RMSE = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (r_{u,i} - \hat{r}_{u,i})^2} \quad (3)$$

— **MAE** (Mean Absolute Error) :

$$MAE = \frac{1}{|T|} \sum_{(u,i) \in T} |r_{u,i} - \hat{r}_{u,i}| \quad (4)$$

3.6.2 Métriques de Recommandation

— **Precision@k** : proportion d'items pertinents dans les k premiers recommandés

$$Precision@k = \frac{|\text{Recommandés} \cap \text{Pertinents}|}{k} \quad (5)$$

— **Recall@k** : proportion d'items pertinents capturés parmi tous les pertinents

$$Recall@k = \frac{|\text{Recommandés} \cap \text{Pertinents}|}{|\text{Pertinents}|} \quad (6)$$

Items pertinents : ceux avec note ≥ 4 dans le test set.

4 Résultats Expérimentaux

4.1 Statistiques du Dataset Après Prétraitement

Métrique	Initial	Final
Nombre d'interactions	82,790	16,046
Nombre d'utilisateurs	47,958	2,271
Nombre d'items	1,378	302
Densité de la matrice	0.125%	1.74%
Pourcentage conservé	100%	19.38%

TABLE 1 – Évolution des statistiques après prétraitement

Malgré le filtrage agressif, la sparsité reste très élevée (98.26%), ce qui représente un défi majeur pour le filtrage collaboratif.

Interprétation : Le filtrage converge en 6 itérations. On passe de 82,790 à 16,046 interactions (19.4% conservé). La densité s'améliore à 1.74% mais la sparsité reste très élevée (98.26%), ce qui posera des défis importants pour le filtrage collaboratif.

4.2 Distribution des Données

- **Interactions par utilisateur :** moyenne = 7.07, médiane = 6.0, min = 5, max = 26
- **Interactions par item :** moyenne = 53.13, médiane = 30.5, min = 5, max = 449

La distribution montre une forte asymétrie avec de nombreux utilisateurs ayant exactement 5 interactions (le minimum filtré) et quelques items très populaires.

4.3 Performances du Filtrage Collaboratif

Métrique	Valeur
RMSE	1.0394
MAE	0.7153
Precision@10	0.0054 (0.54%)
Recall@10	0.0315 (3.15%)
Temps d'évaluation	~18 minutes

TABLE 2 – Résultats du filtrage collaboratif item-based k-NN

Résultats CF : RMSE de 1.04 et MAE de 0.72 indiquent une erreur moyenne inférieure à 1 étoile sur les prédictions. Cependant, Précision@10 de 0.54% et Recall@10 de 3.15% montrent des performances faibles en recommandation, dues à la sparsité extrême des données.

4.4 Performances du Filtrage par Contenu

Métrique	Valeur
Precision@10	0.0365 (3.65%)
Recall@10	0.2680 (26.80%)
Temps d'évaluation	~1.5 secondes

TABLE 3 – Résultats du filtrage basé sur le contenu TF-IDF

Résultats Contenu : Précision@10 de 3.65% et Recall@10 de 26.80%, nettement supérieurs au CF. Cette performance s'explique par l'exploitation efficace des attributs structurés (catégorie, fit) qui sont bien renseignés dans le dataset.

4.5 Comparaison des Modèles

Métrique	CF	Contenu	Amélioration
Precision@10	0.54%	3.65%	+575%
Recall@10	3.15%	26.80%	+750%
Temps d'exécution	18 min	1.5 s	720× plus rapide

TABLE 4 – Comparaison des performances relatives

Le filtrage par contenu surpasse significativement le filtrage collaboratif sur toutes les métriques de recommandation.

5 Analyse et Interprétation

5.1 Pourquoi le Contenu Gagne

Constat principal : Le filtrage par contenu surpasse largement le CF sur ce dataset. La précision est $6.8\times$ meilleure (3.65% vs 0.54%) et le recall $8.5\times$ meilleur (26.80% vs 3.15%).

Explication : La sparsité extrême (98.26%) limite fortement le CF qui manque d'overlaps entre utilisateurs pour calculer des similarités robustes. Avec seulement 7 interactions en moyenne par utilisateur, le CF peine à identifier des voisins pertinents.

À l'inverse, le contenu exploite efficacement les attributs structurés (catégorie, fit) qui sont bien renseignés même quand les avis textuels sont courts. Pour la mode, ces attributs sont très informatifs : un utilisateur qui aime des robes bien ajustées appréciera probablement d'autres robes similaires.

5.2 Limites du CF

Le k-NN item-based utilisé est trop basique pour des données sparses. Des méthodes de factorisation matricielle (SVD, ALS) auraient mieux capturé les patterns latents. De plus, l'évaluation prend 18 minutes contre 1.5 secondes pour le contenu, rendant le CF

impraticable en production. Enfin, le CF souffre du problème du cold start (nouveaux utilisateurs/items), contrairement au contenu.

5.3 Performances Absolues

Même le meilleur modèle (contenu) n'atteint que 3.65% de précision : moins d'1 recommandation sur 10 est pertinente. Cela reste modeste à cause de la sparsité résiduelle (98.26%), des avis textuels courts, et du manque de features riches (images, prix, matériaux). Pour la production, un modèle hybride serait nécessaire.

6 Problèmes Rencontrés

Sparsité écrasante : Le dataset initial était quasi vide (99.87% de sparsité). Filtrer les utilisateurs/items avec moins de 5 interactions améliore la densité à 1.74%, mais fait perdre 80% des données. Compromis nécessaire mais douloureux.

Avis textuels courts : Beaucoup d'avis vides ou incomplets. Solution : amplifier le poids des attributs structurés (catégorie $\times 3$, fit $\times 2$) dans les vecteurs TF-IDF pour compenser.

CF trop lent : 18 minutes pour évaluer 4,033 prédictions. Même optimisé avec matrices sparse et k-NN brute force, le CF reste impraticable pour un usage temps réel.

7 Pistes d'Amélioration

SVD au lieu de k-NN : La factorisation matricielle découvre des facteurs latents (style casual, formel, sportif) qui expliquent mieux les préférences avec peu de données.

Enrichir les features : Ajouter images (CNN), descriptions complètes, prix et matériaux pour donner plus d'informations aux modèles.

Modèle hybride : Combiner CF et contenu ($score = 0.3 \times CF + 0.7 \times contenu$) pour bénéficier des forces des deux approches.

Embeddings avancés : BERT pour mieux comprendre les avis textuels, et modèles séquentiels pour capturer l'ordre des achats.

8 Conclusion

8.1 Résultats Principaux

Sur le dataset ModCloth avec sa sparsité extrême (98.26%), le **filtrage par contenu surpasse largement le filtrage collaboratif** :

- Recall : 26.80% vs 3.15% ($8.5\times$ meilleur)
- Précision : 3.65% vs 0.54% ($6.8\times$ meilleure)

Explication : Le CF manque d'interactions communes entre utilisateurs pour calculer des similarités robustes. Le contenu exploite efficacement les attributs structurés (catégorie, ajustement) qui compensent les avis textuels souvent courts.

8.2 Statistiques Finales du Projet

Statistique	Valeur
Données initiales	82,790 interactions
Données finales	16,046 interactions (19.38%)
Utilisateurs	2,271
Items (vêtements)	302
Densité de la matrice	1.74%
Sparsité	98.26%
Filtrage Collaboratif :	
RMSE	1.04
MAE	0.72
Precision@10	0.54%
Recall@10	3.15%
Temps d'évaluation	~18 minutes
Filtrage Contenu :	
Precision@10	3.65%
Recall@10	26.80%
Temps d'évaluation	~1.5 secondes

TABLE 5 – Résumé complet des performances du projet

8.3 Limites Identifiées

- **Performances absolues modestes** : Précision maximale de seulement 3.65%
- **Perte de données** : 80% des données supprimées lors du filtrage de sparsité
- **Pas de modèle hybride** : Aucune combinaison CF + contenu testée
- **Avis textuels pauvres** : Beaucoup d'items avec descriptions incomplètes

Ce projet démontre l'importance de choisir la bonne approche en fonction des caractéristiques du dataset. Le filtrage collaboratif, bien que populaire, n'est pas adapté à tous les contextes, particulièrement quand les données sont extrêmement sparses et que des métadonnées riches sont disponibles.